多线程与并发原理

© 硬核空间

为什么需要多线程

- CPU：你们都慢！死！了！
- 现代CPU都是多核的
- Java的执行模型是同步/阻塞(block)的
- 默认情况下只有一个线程
  - 处理问题非常自然
  - 但是具有严重的性能问题

© 硬核空间

我能不能多请几个人来干活?

© 硬核空间

开启一个新的线程

- Thread
  - Java中只有这么一种东西代表线程
  - start方法才能并发执行！
  - 每多开一个线程，就多一个执行流
  - 方法栈(局部变量)是线程私有的
  - 静态变量/类变量是被所有线程共享的

© 硬核空间

线程难的本质原因是
你要看着同一份代码，
想象不同的人在疯狂地以乱序执行它

© 硬核空间

演示多线程带来的性能提升

- 对于IO密集型应用极其有用
  - 网络IO（通常包括数据库）
  - 文件IO
- 对于CPU密集型应用稍有折扣
- 性能提升的上限在哪里?
  - 单核CPU 100%
  - 多核CPU N*100%

© 硬核空间

昂贵的线程

- 能不能使用线程达到无穷无尽的性能提升?
  - 十个女人一个月能生一个孩子么?
- 线程的昂贵性在于
  - 第一，CPU切换上下文很慢
  - 第二，线程需要占用内存等系统资源
- 如果你的应用一天才几个用户
  - new Thread().start()
- 如果你的应用负载很高
  - 使用线程池:JUC包

© 硬核空间

线程安全

- 你享用了多线程的便利，就要付出代价
  - 原子性
  - 共享变量
  - 默认的实现几乎都不是线程安全的！

© 硬核空间

线程不安全的表现

- 数据错误
  - i++
  - if–then–do

© 硬核空间

线程不安全的表现

- 死锁

© 硬核空间

线程不安全的表现

- 著名的HashMap的死循环问题
- 写一段代码来重现死锁
- 预防死锁产生的原则：
  - 所有的线程都按照相同的顺序获得资源的锁
- 死锁问题的排查
- 多线程的经典问题：哲学家用餐

© 硬核空间

线程安全

- 实现线程安全的基本手段
  - 不可变类
    - Integer/String/....

© 硬核空间

线程安全

- 实现线程安全的基本手段
  - synchronized同步块
  - 同步块同步了什么东西?
    - synchronized(一个对象) 把这个对象当成锁
    - Static synchronized方法 把Class对象当成锁
    - 实例的synchronnized方法把该实例当成锁
  - Collections.synchronized

© 硬核空间

线程安全

- 实现线程安全的基本手段
  - JUC包
    - AtomicInteger/...
    - ConcurrentHashMap
      - 任何使用HashMap有线程安全问题的地方
      - 都无脑地使用ConcurrentHashMap替换即可。
    - ReentrantLock
    - ...

© 硬核空间

Object类里的线程方法

- 线程的历史
  - Java从一开始就把线程作为语言特性，提供语言级的支持
- 为什么Java中的所有对象都可以成为锁?
  - Object.wait()/notify()/notifyAll()方法
  - 线程的状态与线程调度
  - 【扩展】为什么说Java的线程

© 硬核空间

多线程的经典问题
生产者/消费者模型

- 使用三种方法来解决它
  - wait/notify/notifyAll
  - Lock/Condition
  - BlockingQueue

© 硬核空间

线程池与Callable/Future

- 什么是线程池
  - 线程是昂贵的（Java线程模型的缺陷）
  - 线程池是预先定义好的若干个线程
  - Java中的线程池
- Callable/Future
  - 类比Runnable，Callable可以返回值，抛出异常
  - Future代表一个“未来才会返回的结果”
- 实战：多线程的WordCount

© 硬核空间